10 SQL Queries Every Data Analyst Must Know

> Most analyst questions reduce to ten query shapes; mastering how the database *executes* them turns SQL from a guessing game into a reliable instrument.

Audience: developers and analysts who can read basic SQL but freeze when asked for "top 3 products per category" or "month-over-month retention" · **Prereqs:** know what a table, row, and column are; have run at least one `SELECT` · **Read time:** 22 min

TL;DR

- SQL is **declarative**: you describe the result, the planner decides the steps. The 10 queries below are 10 *result shapes* that cover ~90% of analyst work.
- `WHERE` filters rows before grouping; `HAVING` filters groups after aggregation. They are not interchangeable.
- `JOIN` is set algebra on keys, not a loop. Inner, left, and full outer differ only in which non-matching rows survive.
- `GROUP BY` collapses many rows into one per group; **window functions** keep every row but add a per-group computation alongside it.
- **CTEs** (`WITH`) name intermediate results so a query reads top-to-bottom like a pipeline instead of inside-out like nested parentheses.
- Top-N-per-group, cohort retention, and percentile reports all follow the same pattern: partition, rank or bucket, then filter.

The mental model (start here)

A relational database is a warehouse of labeled boxes (rows) sorted into shelves (tables). A query is a shopping list with rules: "find every box on shelf A whose label matches a box on shelf B, count them by color, and only show colors with more than five boxes." You never tell the warehouse *how* to walk the aisles. You describe the haul. The planner picks the route.

Ten query shapes cover most analyst questions because most business questions reduce to: *filter, combine, group, rank, or compare over time*. Everything else is a variation on those five verbs.

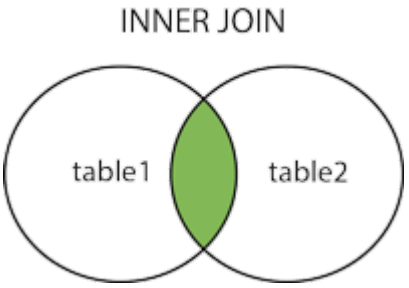
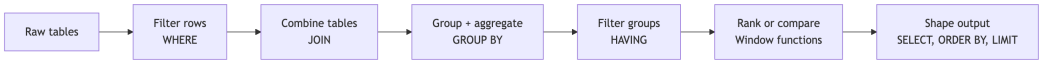


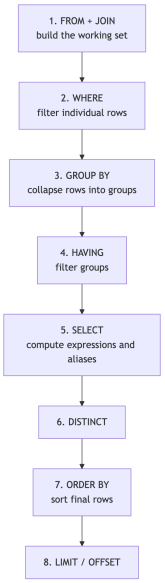
Figure 1: Inner join as set intersection on a key. Only rows whose key appears in both sides survive. Source: Wikipedia, "Join (SQL)".



> 🧠 Check: before reading on, predict which clause runs first when SQL processes `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT`. Most people guess wrong.

Step 1: The logical execution order (why SQL feels backwards)

You write `SELECT` first, but the database evaluates it almost last. The **logical execution order** is the conceptual sequence the planner follows to produce a result.



Two consequences fall out of this order:

1. You cannot reference a `SELECT` alias inside `WHERE`, because `WHERE` runs before `SELECT` evaluates expressions. You *can* reference it inside `ORDER BY`, which runs after.
2. Aggregate functions like `COUNT(*)` are illegal inside `WHERE`, because grouping has not happened yet. Use `HAVING` for that.

We will use one running dataset for every example.

```sql
-- Sample tables used throughout this explainer
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  name        TEXT,
  country     TEXT,
  signup_date DATE
);

CREATE TABLE orders (
  order_id    INT PRIMARY KEY,
  customer_id INT REFERENCES customers(customer_id),
  order_date  DATE,
  amount      NUMERIC(10,2),
  status      TEXT  -- 'paid', 'refunded', 'pending'
);

INSERT INTO customers VALUES
  (1, 'Ada',    'UK', '2025-01-10'),
  (2, 'Bo',     'US', '2025-02-03'),
  (3, 'Chen',   'SG', '2025-02-20'),
  (4, 'Dara',   'US', '2025-03-15'),
  (5, 'Eli',    'UK', '2025-04-01');

INSERT INTO orders VALUES
  (101, 1, '2025-02-01', 120.00, 'paid'),
  (102, 1, '2025-03-05',  80.00, 'paid'),
  (103, 2, '2025-03-08', 200.00, 'paid'),
  (104, 2, '2025-04-10',  50.00, 'refunded'),
  (105, 3, '2025-04-12', 300.00, 'paid'),
  (106, 4, '2025-04-20',  75.00, 'paid'),
  (107, 4, '2025-05-01',  90.00, 'pending'),
  (108, 1, '2025-05-15', 150.00, 'paid');
```

Query 1, the foundation: `SELECT` with `WHERE`.

```sql
-- "Paid orders above $100, newest first, top 5"
SELECT order_id, customer_id, amount, order_date
FROM   orders
WHERE  status = 'paid' AND amount > 100
ORDER BY order_date DESC
LIMIT  5;
```

Result (using the seed data):

order_id	customer_id	amount	order_date
108	1	150.00	2025-05-15
105	3	300.00	2025-04-12

103	2	200.00	2025-03-08
101	1	120.00	2025-02-01

> 🧠 Check: if you change `WHERE amount > 100` to `WHERE amount > AVG(amount)`, what happens? (It errors. Aggregates require a subquery or a `HAVING` clause, because `WHERE` runs before aggregation.)

Step 2: JOINs as set operations on keys

Query 2 is the join. A **join** matches rows from two tables wherever a key expression is true. It is not a loop; it is set algebra. Four flavors matter:

- **INNER JOIN** keeps only rows where the key appears in both sides.
- **LEFT JOIN** keeps every row from the left side, fills `NULL` where the right side has no match.
- **RIGHT JOIN** is the mirror, rarely used because you can swap table order.
- **FULL OUTER JOIN** keeps every row from both sides, `NULL` filling the gaps.

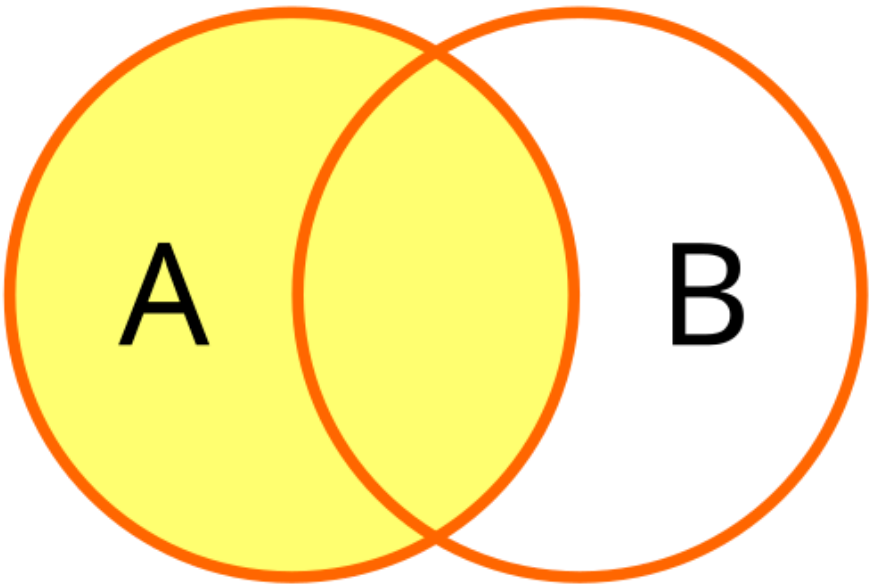
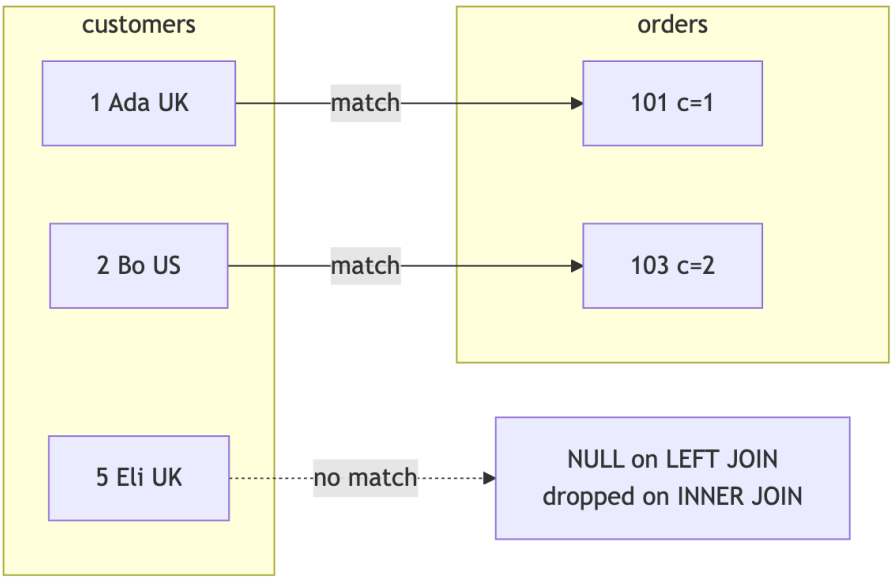


Figure 2: Left join keeps all rows from A and pads with NULLs where B has no match. Source: Wikipedia, "Join (SQL)".




```sql
-- Customer names with their paid order amounts.
-- LEFT JOIN so customers with zero orders still appear.
SELECT c.name,
       c.country,
       o.order_id,
       o.amount
FROM   customers c
LEFT JOIN orders o
       ON o.customer_id = c.customer_id
      AND o.status      = 'paid'
ORDER BY c.name, o.order_date;
```

The condition `o.status = 'paid'` sits inside the `ON` clause, not `WHERE`. If you move it to `WHERE`, you silently convert the left join into an inner join, because the `NULL` rows for customers with no paid orders fail the `status = 'paid'` test and get dropped. This is the single most common SQL bug.

> 🧠 Check: customer Eli has no orders. With the query above, how many rows does Eli produce? (Answer: exactly one, with `NULL` in `order_id` and `amount`.)

Step 3: GROUP BY and aggregates collapse rows

Query 3 is aggregation. **GROUP BY** partitions rows into buckets that share the same value for the listed columns, then runs aggregate functions per bucket. The output has one row per bucket.

```sql
-- Revenue and order count per country (paid only)
SELECT c.country,
       COUNT(*)          AS paid_orders,
       SUM(o.amount)     AS revenue,
       AVG(o.amount)::numeric(10,2) AS avg_ticket
FROM   customers c
JOIN   orders    o ON o.customer_id = c.customer_id
WHERE  o.status = 'paid'
GROUP  BY c.country
ORDER  BY revenue DESC;
```

country	paid_orders	revenue	avg_ticket
UK	3	350.00	116.67
SG	1	300.00	300.00
US	2	275.00	137.50

Three aggregates you will reach for daily:

- `COUNT(*)` counts rows, including `NULL`s.
- `COUNT(col)` counts non-`NULL` values of `col`.
- `COUNT(DISTINCT col)` counts unique non-`NULL` values, and is much slower because it must deduplicate.

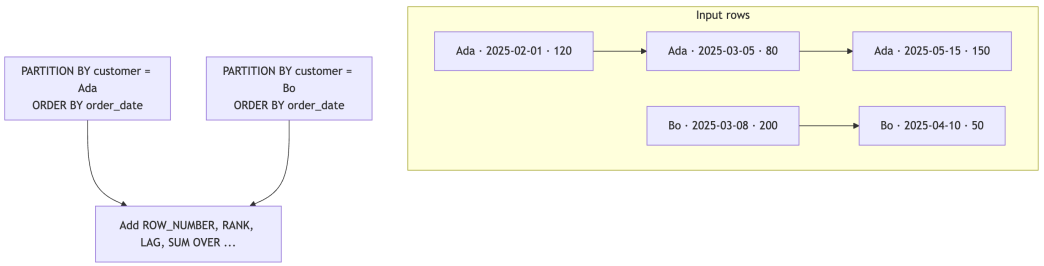
Query 4 is `HAVING`, the filter that runs *after* grouping.

```sql
-- Countries with at least 2 paid orders AND average ticket above $100
SELECT c.country,
       COUNT(*)      AS paid_orders,
       AVG(o.amount) AS avg_ticket
FROM   customers c
JOIN   orders    o ON o.customer_id = c.customer_id
WHERE  o.status = 'paid'           -- per-row filter
GROUP  BY c.country
HAVING COUNT(*)      ⩾ 2
   AND AVG(o.amount) > 100;        -- per-group filter
```

Rule of thumb: if your filter mentions an aggregate, it belongs in `HAVING`. Otherwise it belongs in `WHERE`, where it runs earlier and on fewer rows.

Step 4: Window functions keep every row

Query 5 introduces **window functions**, which compute a value over a "window" of related rows *without* collapsing them. The row count of the output equals the row count of the input. The window is defined by `OVER (PARTITION BY ... ORDER BY ...)`.



The four window functions you will use 80% of the time:

- `ROW_NUMBER()` assigns 1, 2, 3, ... within each partition. Ties broken by `ORDER BY`.
- `RANK()` assigns the same rank to ties and leaves gaps (1, 2, 2, 4).
- `DENSE_RANK()` ranks ties but skips no numbers (1, 2, 2, 3).
- `LAG(col, n)` and `LEAD(col, n)` peek at the prior or next row in the partition.

Query 6, the classic "top N per group" pattern, uses `ROW_NUMBER`:

```
-- Top 1 paid order per customer (largest amount, tie-break newest first)
WITH ranked AS (
  SELECT o.*,
         ROW_NUMBER() OVER (
           PARTITION BY o.customer_id
           ORDER BY o.amount DESC, o.order_date DESC
         ) AS rn
  FROM   orders o
  WHERE  o.status = 'paid'
)
SELECT customer_id, order_id, amount, order_date
FROM   ranked
WHERE  rn = 1;
```

This pattern (rank inside a CTE, then filter on the rank) is the standard solution for "top N per category." Trying to do it with `GROUP BY` and `MAX` loses the other columns of the winning row.

Query 7 uses `LAG` for period-over-period comparisons:

```
-- Each paid order vs the customer's previous paid order
SELECT customer_id,
       order_date,
       amount,
       LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_amount
       amount - LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS de
FROM   orders
WHERE  status = 'paid';
```

> 🧠 Check: if a customer has only one paid order, what is `prev_amount`? (It is `NULL`, because there is no prior row in the partition. `delta` is therefore also `NULL`. Use `COALESCE(prev_amount, 0)` only if zero is the meaningful default.)

Step 5: CTEs, subqueries, and recursion

Query 8 is the **CTE** (Common Table Expression), introduced by `WITH`. A CTE is a named, single-use temporary result set that makes a multi-step query readable top to bottom.

```
WITH paid AS (
  SELECT * FROM orders WHERE status = 'paid'
),
by_customer AS (
  SELECT customer_id, SUM(amount) AS lifetime_value
  FROM   paid
  GROUP  BY customer_id
)
SELECT c.name, c.country, b.lifetime_value
FROM   customers c
JOIN   by_customer b ON b.customer_id = c.customer_id
ORDER  BY b.lifetime_value DESC;
```

Subqueries do the same job but inside-out, and they get unreadable past two levels of nesting. Modern planners (PostgreSQL 12+, recent SQL Server, BigQuery, Snowflake) inline CTEs by default, so the readability win is free.

Recursive CTEs handle hierarchies and sequences. Example: generate a continuous date series so you can spot days with zero orders.

```
WITH RECURSIVE dates(d) AS (
  SELECT DATE '2025-02-01'
  UNION ALL
  SELECT d + INTERVAL '1 day'
  FROM   dates
  WHERE  d < DATE '2025-05-31'
)
SELECT d AS day,
       COUNT(o.order_id) AS orders_that_day
FROM   dates
LEFT JOIN orders o ON o.order_date = dates.d
GROUP  BY d
ORDER  BY d;
```

The recursive part has two halves joined by `UNION ALL`: an **anchor** (the seed row) and a **recursive step** that references the CTE itself until the `WHERE` condition stops it. Always include a stop condition. An infinite recursion will be killed by the server but burn resources first.

Step 6: CASE WHEN, pivoting, and date bucketing

Query 9 reshapes data with `CASE WHEN`. This is how you bucket continuous values and pivot long-format data into wide format without a dedicated pivot operator.

```sql
-- Monthly paid revenue, pivoted into one column per status
SELECT date_trunc('month', order_date)::date AS month,
       SUM(CASE WHEN status = 'paid'     THEN amount ELSE 0 END) AS paid_rev,
       SUM(CASE WHEN status = 'refunded' THEN amount ELSE 0 END) AS refunded_rev,
       SUM(CASE WHEN status = 'pending'  THEN amount ELSE 0 END) AS pending_rev,
       COUNT(*)                                                  AS total_orders
FROM   orders
GROUP  BY 1
ORDER  BY 1;
```

month	paid_rev	refunded_rev	pending_rev	total_orders
2025-02-01	120.00	0.00	0.00	1
2025-03-01	280.00	0.00	0.00	2
2025-04-01	375.00	50.00	0.00	3
2025-05-01	150.00	0.00	90.00	2

`date_trunc('month', x)` is the **date bucket**: it floors a timestamp to the start of its month. Equivalents elsewhere: `DATE_FORMAT(x, '%Y-%m-01')` in MySQL, `DATEFROMPARTS(YEAR(x), MONTH(x), 1)` in SQL Server, `TIMESTAMP_TRUNC(x, MONTH)` in BigQuery.

> 🧠 Check: why bucket by `date_trunc('month', order_date)` instead of `EXTRACT(MONTH FROM order_date)`? (The latter collapses 2024-03 and 2025-03 into the same bucket. Always bucket on a date or timestamp, not on a month number alone, unless you also group by year.)

Step 7: Percentiles and cohort retention

Query 10 is the percentile / cohort pattern. **Percentiles** use `PERCENTILE_CONT` (linear interpolation) or `PERCENTILE_DISC` (exact value). **Cohort retention** groups users by their signup period and measures activity in each subsequent period.

```sql
-- Cohort retention: % of each signup-month cohort that ordered in each later month
WITH cohort AS (
  SELECT customer_id,
         date_trunc('month', signup_date)::date AS cohort_month
  FROM   customers
),
activity AS (
  SELECT o.customer_id,
         date_trunc('month', o.order_date)::date AS active_month
  FROM   orders o
  WHERE  o.status = 'paid'
  GROUP  BY 1, 2
),
joined AS (
  SELECT c.cohort_month,
         a.active_month,
         (EXTRACT(YEAR  FROM a.active_month) - EXTRACT(YEAR  FROM c.cohort_month)) *
       + (EXTRACT(MONTH FROM a.active_month) - EXTRACT(MONTH FROM c.cohort_month)) AS
         c.customer_id
  FROM   cohort c
  JOIN   activity a ON a.customer_id = c.customer_id
),
sizes AS (
  SELECT cohort_month, COUNT(*) AS cohort_size
  FROM   cohort
  GROUP  BY cohort_month
)
SELECT j.cohort_month,
       j.month_offset,
       COUNT(DISTINCT j.customer_id)                      AS active_users,
       s.cohort_size,
       ROUND(100.0 * COUNT(DISTINCT j.customer_id) / s.cohort_size, 1) AS retention_
FROM   joined j
JOIN   sizes  s ON s.cohort_month = j.cohort_month
GROUP  BY j.cohort_month, j.month_offset, s.cohort_size
ORDER  BY j.cohort_month, j.month_offset;
```

And the percentile sibling, useful for SLA and pricing analysis:

```sql
-- Median, P90, P99 order value per country (paid only)
SELECT c.country,
       PERCENTILE_CONT(0.5)  WITHIN GROUP (ORDER BY o.amount) AS p50,
       PERCENTILE_CONT(0.90) WITHIN GROUP (ORDER BY o.amount) AS p90,
       PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY o.amount) AS p99
FROM   customers c
JOIN   orders    o ON o.customer_id = c.customer_id
WHERE  o.status = 'paid'
GROUP  BY c.country;
```

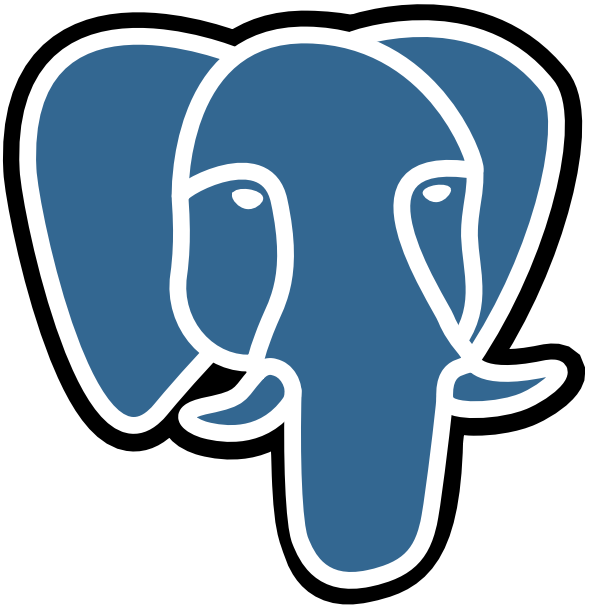


Figure 3: PostgreSQL is the reference dialect used throughout this explainer. `PERCENTILE_CONT`*,* `date_trunc`*, and recursive CTEs are SQL standard features supported by Postgres, Snowflake, BigQuery, Redshift, and recent SQL Server. MySQL added window functions in 8.0 and* `PERCENTILE_CONT` *is still missing as of 8.4. Source: Wikimedia Commons.*

Worked example: end to end

Question: "For each country, what is the second-largest paid order placed in the most recent calendar month that had at least 2 paid orders?"

Translate the question into the pipeline:

1. Filter to paid orders only.
2. Bucket by month.
3. Per country and month, count paid orders. Keep only `(country, month)` pairs with `⩾ 2` paid orders.
4. Within each country, pick the most recent qualifying month.
5. Within that `(country, month)` slice, rank paid orders by amount desc and pick `rn = 2`.

```sql
WITH paid AS (
  SELECT o.*,
         c.country,
         date_trunc('month', o.order_date)::date AS month
  FROM   orders    o
  JOIN   customers c ON c.customer_id = o.customer_id
  WHERE  o.status = 'paid'
),
qualifying_months AS (
  SELECT country, month, COUNT(*) AS n
  FROM   paid
  GROUP  BY country, month
  HAVING COUNT(*) ⩾ 2
),
latest_per_country AS (
  SELECT country, MAX(month) AS month
  FROM   qualifying_months
  GROUP  BY country
),
ranked AS (
  SELECT p.country, p.month, p.order_id, p.amount,
         ROW_NUMBER() OVER (PARTITION BY p.country
                            ORDER BY p.amount DESC, p.order_id) AS rn
  FROM   paid p
  JOIN   latest_per_country l
         ON l.country = p.country AND l.month = p.month
)
SELECT country, month, order_id, amount
FROM   ranked
WHERE  rn = 2;
```

Trace with the seed data:

- `paid` has 6 rows after filtering to `status = 'paid'`.
- Grouped by `(country, month)`: UK has `(Feb, 1)`, `(Mar, 1)`, `(May, 1)`; US has `(Mar, 1)`, `(Apr, 1)`; SG has `(Apr, 1)`. **No `(country, month)` reaches 2 paid orders.**
- `qualifying_months` is empty, so the final result set is empty.

Output: zero rows. This is the right answer for this dataset, and it illustrates an important habit: when a query returns nothing, the bug is often in the data, not the SQL. Run each CTE in isolation to see where rows disappear.

Add two rows for Eli (`INSERT ... (109, 5, '2025-05-20', 60.00, 'paid'), (110, 5, '2025-05-25', 40.00, 'paid')`). UK now has three paid orders in May (Ada's 150.00, Eli's 60.00 and 40.00). The second-largest is order 109 at 60.00.

Output: `(UK, 2025-05-01, 109, 60.00)`.

Where the analogy breaks down

- **Analogy 1 (warehouse and shopping list):** holds for the declarative idea (you describe the haul, not the route) and for set-based thinking. It breaks for window functions, which behave less like "pick boxes off shelves" and more like "walk down a sorted aisle taking notes about neighbors." Window functions are inherently order-sensitive within a partition; the warehouse metaphor implies unordered sets.
- **Analogy 2 (JOIN as set intersection via Venn diagram):** holds for the *which rows survive* question. It breaks for cardinality: a join is not just intersection, it is a **Cartesian product** restricted by the `ON` condition. If one customer has 4 paid orders, a `customers JOIN orders` produces 4 rows for that customer, not 1. Venn diagrams hide this row multiplication and trip up beginners who expected "one row per customer."

Common misconceptions

- ❌ "`COUNT(*)` and `COUNT(col)` are the same." Actually, `COUNT(*)` counts every row, `COUNT(col)` counts only rows where `col IS NOT NULL`. If `col` is nullable, the two return different numbers and a confused stakeholder.
- ❌ "Use `WHERE` for everything; `HAVING` is just an older keyword." Actually, `WHERE` filters rows before `GROUP BY`, `HAVING` filters groups after. Putting `HAVING SUM(amount) > 1000` inside `WHERE` is a syntax error in standard SQL because the aggregate does not exist yet.
- ❌ "`SELECT DISTINCT` is a cheap way to dedupe." Actually, `DISTINCT` forces the planner to sort or hash the entire result set, which is often more expensive than fixing the underlying join cardinality. If you find yourself writing `SELECT DISTINCT` to clean up a join, the join condition is probably wrong (often a missing key on a one-to-many table).
- ❌ "`LEFT JOIN` plus a `WHERE` filter on the right table is fine." Actually, putting `WHERE right_table.col = 'x'` after a left join silently drops the `NULL`-padded rows and turns the query into an inner join. Move the predicate into the `ON` clause if you want to preserve unmatched left rows.

What it's bad at

- **Graph traversal beyond a few hops.** Recursive CTEs can walk a graph, but depth-N queries blow up combinatorially and most engines have a recursion limit. Use a graph database (Neo4j, Memgraph) or a graph extension if you need shortest paths or PageRank.
- **Iterative computation.** Pricing engines that loop until convergence, Monte Carlo, and ML training do not belong in SQL. Pull the data out, compute in Python, push results back.
- **Schema-less or deeply nested JSON.** SQL engines have JSON operators (`→`, `⟶`, `JSONB_PATH_QUERY` in Postgres), but if most of your data is nested JSON, a document store or a columnar format with nested types (Parquet, BigQuery STRUCT) will be faster and clearer.
- **Real-time row-level reactivity.** SQL queries are pull-based snapshots. For "notify me when X changes," use change-data-capture, triggers, or a streaming engine (Kafka, Materialize, Flink).

Glossary

- **Aggregate function** — many rows in, one value per group out (`SUM`, `AVG`, `COUNT`).
- **Anchor (recursive CTE)** — the non-recursive seed query that starts a recursive CTE.
- **Cartesian product** — every row of A paired with every row of B, the implicit starting point of any join before the `ON` condition filters it.
- **Cohort retention** — the percentage of users from a signup period who remain active in each subsequent period.
- **CTE (Common Table Expression)** — a named, query-scoped temporary result set introduced with `WITH name AS (...)`.
- **Date bucket** — a date or timestamp floored to a fixed period such as month or week, used to group time-series data.
- **Declarative** — describing the result rather than the procedure; SQL is declarative because the planner chooses the execution strategy.
- **DENSE_RANK** — window function ranking ties equally without leaving gaps in the sequence.
- **Full outer join** — a join that keeps unmatched rows from both sides, padding with `NULL`s.
- **GROUP BY** — clause that partitions rows by shared column values so aggregates compute per partition.
- **HAVING** — clause that filters groups after aggregation, the post-aggregation analog of `WHERE`.
- **Inner join** — a join that keeps only rows whose key appears in both tables.
- **LAG / LEAD** — window functions that return a value from a row offset before or after the current row within the partition.
- **Left join** — a join that keeps every row from the left table, `NULL`-padding the right side when there is no match.
- **Logical execution order** — the conceptual sequence (FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT) the planner uses to evaluate a query.
- **Partition (window function)** — the subset of rows defined by `PARTITION BY` over which a window function computes.
- **PERCENTILE_CONT** — aggregate that returns a continuous (interpolated) percentile of a sorted column.
- **Pivot** — reshaping long-format data (one row per category) into wide format (one column per category), typically with `CASE WHEN` plus `SUM`.
- **RANK** — window function ranking ties equally and leaving gaps in the sequence.
- **Recursive CTE** — a CTE whose definition references itself, used for hierarchies and series generation.
- **ROW_NUMBER** — window function assigning a unique integer per row within a partition, breaking ties with the `ORDER BY` clause.
- **Window function** — a function that computes over a set of related rows defined by `OVER (...)` without collapsing the row count.

Further reading

- PostgreSQL official documentation, "Queries" chapter — read this for the authoritative reference on `WITH`, window functions, and `PERCENTILE_CONT`. https://www.postgresql.org/docs/current/queries.html
- PostgreSQL official documentation, "Window Functions Tutorial" — read this for the clearest hands-on intro to `OVER (PARTITION BY ... ORDER BY ...)`. https://www.postgresql.org/docs/current/tutorial-window.html
- Mode Analytics SQL Tutorial — read this for a free, browser-based interactive course covering joins, aggregates, and window functions on real datasets. https://mode.com/sql-tutorial
- Use The Index, Luke! by Markus Winand — read this for how indexes and execution plans actually drive query performance across Postgres, MySQL, SQL Server, and Oracle. https://use-the-index-luke.com
- Wikipedia article "Join (SQL)" — read this for the canonical Venn diagrams of inner, left, right, and full outer joins. https://en.wikipedia.org/wiki/Join_(SQL)
- Joe Celko, "SQL for Smarties" — read this for set-based thinking patterns that turn procedural-style SQL into idiomatic relational queries.
- Itzik Ben-Gan, "T-SQL Window Functions" — read this for the deepest treatment of window function semantics and performance, dialect-agnostic despite the title.
- SQLZoo — read this for short, graded exercises that drill `JOIN`, `GROUP BY`, and nested queries until they are automatic. https://sqlzoo.net
- DB Fiddle and SQL Fiddle — read this for paste-and-run sandboxes to test queries across Postgres, MySQL, SQL Server, and SQLite. https://www.db-fiddle.com
- Snowflake documentation, "Analytic Functions" — read this for cloud-warehouse-flavored window function recipes including frame clauses (`ROWS BETWEEN ...`). https://docs.snowflake.com/en/sql-reference/functions-analytic
- Google BigQuery documentation, "Standard SQL Query Syntax" — read this for the dialect powering most modern analytics stacks and a clean spec of array and struct operations. https://cloud.google.com/bigquery/docs/reference/standard-sql/query-syntax